MATH0499: Projet

Saïd Hormat-Allah, Jamaà Jair

Table des matières

1 Introduction 3

2 Choix opérés 3
- 2.1 Format de fichier . . . 3
- 2.2 Architecture du projet . . . 3
  - 2.2.1 main.py . . . 3
  - 2.2.2 file.py . . . 3
  - 2.2.3 utils.py . . . 3
  - 2.2.4 generator.py . . . 3

3 Difficultés rencontrés 4
- 3.1 Création de graphe fortement connexe . . . 4
- 3.2 Le parcours aléatoire de l'agent dans le graphe . . . 4
- 3.3 La distribtion des probabilités pour la liste des successeurs de v . . . 4

4 Description des fonctions 4
- 4.1 Lecture du graphe . . . 4
- 4.2 Estimation de revenir en v . . . 4
  - 4.2.1 Avec un chemin exactement n . . . 5
  - 4.2.2 Avec un chemin au plus n . . . 5
- 4.3 Parcours aléatoire du graphe . . . 5
- 4.4 Taille du chemin le plus grand/petit . . . 5
- 4.5 Statistiques . . . 5
  - 4.5.1 Effectif total . . . 5
  - 4.5.2 Distribution des fréquences . . . 6
  - 4.5.3 La moyenne des chemins . . . 6
  - 4.5.4 La variance des chemins . . . 6

5 Données de retour du programme 6

6 Utilisation du programme 7

1 Introduction

Étant donné un graphe fortement connexe fourni en entrée (possibilité de charger un fichier, fournir un fichier exemple pour un graphe de 100 sommets), l'utilisateur sélectionne un sommet de départ v et un entier n. On souhaite estimer la probabilité de revenir en v avec un chemin de longueur exactement n, puis de longueur *au plus* n. On souhaite également compter le nombre de tels chemins. Quand un agent qui parcourt le graphe se trouve en un sommet, on suppose que le choix du successeur se fait de façon uniforme : s'il y a t successeurs, chacun a une probabilité $1/t$ d'être choisi. Expérimenter avec d'autre distributions de probabilités. Fournir des "statistiques", par exemple, sur un échantillon de n chemins, quel est le plus fréquent, etc.

2 Choix opérés

Nous allons traiter les différents choix fait dans le cadre de ce projet.

2.1 Format de fichier

Nous avons choisit le format **JSON** (**J**ava**S**cript **O**bject **N**otation). De par sa simple syntaxe, ce format de fichier permet une facilité de lecture par un humain et peut aussi être facilement écrit par ce dernier. Python offre la possibilité de lire ce format de fichier, grâce à la bibliothèque `json`, en transformant certaines données en dictionnaire (ou liste), ce qui facilite grandement la création de graphe avec la bibliothèque `NetworkX`.

2.2 Architecture du projet

Cette section parlera de la manière dont le projet à été divisé afin d'assurer une bonne lecture du code ainsi qu'une efficacité en terme de développement.

La découpe en fichiers permet une maniabilité du projet et une adaptation aux changement importants dans le projet, par exemple, la lecture de fichier. Et permet aussi une simplicité d'ajout de fonctionnalités sans impacter le reste des implémentations.

2.2.1 main.py

Le fichier `main.py` est le point d'entrée du programme. Permet de "parser" les arguments entrés dans le programme (e.g. le nom de fichier fournit par l'utilisateur, le mode de débogage et s'il faut une autre distribution des probabilités).

2.2.2 file.py

Le fichier `file.py`, le cœur du projet, contient toutes les fonctions importantes du projet. dont la fonction main qui est invoquée dans `main.py`

2.2.3 utils.py

`utils.py` est une bibliothèque qui assure le bon fonctionnement des fonctions utilisés dans `file.py`

2.2.4 generator.py

`generator.py` est mini-programme qui a pour but de générer un graphe fortement connexe avec une entrée utilisateur qui sera le nombre de sommets du graphe.

3 Difficultés rencontrés

3.1 Création de graphe fortement connexe

Lors de la création du graphe fortement connexe, nous étions dans une impasse. Puisque nous arrivions seulement à créer des graphes connexes. Nous respections la règle disant qu'un sommet devait au moins un demi-degré sortant et un demi-degré entrant mais cela n'était pas assez. Après des recherches sur ce qu'est un graphe fortement connexe, nous avons trouvé un écrit [1] sur la création de graphe fortement connexe. Nous étions pas loin de l'algorithme proposé, nous avons oublié de créer un arc entre chaque points v et $v + 1$. Et de relier le dernier sommet au premier.

3.2 Le parcours aléatoire de l'agent dans le graphe

Comme `NetworkX` ne propose pas de fonction pour parcourir aléatoirement le graphe, nous avons du le faire nous même. Cela implique : De parcourir le graphe plus d'une fois à partir du même sommet v, de changer de sommet afin de parcourir le graphe, vérifier si le point choisit possède un demi-degré sortant ou s'il possède un chemin menant vers v. Ceci est un résumé de l'algorithme. Les difficultés étaient, quels devaient être les conditions d'arrêts pour chaque boucles et que devons nous faire pour avoir une liste de chemins complets. Nous avons du faire des boucles infinis imbriqués afin d'être le plus complet possible dans la recherche de chemins allant de v à v.

3.3 La distribtion des probabilités pour la liste des successeurs de v

La bibliothèque `numpy` propose, par sa fonction `np.random.choice(...)`, la possibilité de choisir aléatoirement un élément d'une liste avec une probabilité de $1/t$ pour chaque élément avec t le nombre d'éléments mais aussi une distribution uniforme personnalisée en fournissant une liste de probabilité. Mais pas la possibilité de créer une liste de t éléments avec une distribution personnalisée dont la somme des éléments vaut 1. Nous avons fait un algorithme mais un problème assez important nous bloquait, notre algorithme ne nous donnait pas la valeur 1 (qui est la condition pour que la fonction `np.random.choice` se lance), nos valeurs variaient entre 0,999 et 1,0001. Nous nous sommes donc inspiré par une résolution envoyée dans le site stackoverflow [2] qui est de base fait pour générer une matrice stochastique, nous l'avons adapté pour avoir une simple liste.

4 Description des fonctions

4.1 Lecture du graphe

La fonction `loadGraph()` prend en entrée un seul argument qui est le nom du fichier, cette fonction lit le fichier et retourne un graphe simple orienté.

4.2 Estimation de revenir en v

Afin de calculer le nombre exacte de chemins d'une longueur précise, nous avons utilisé la matrice d'adjacence A de notre graphe. Et nous l'avons augmenté la puissance de la matrice A de n.

1. Maurer 2017.
2. Hangiu 2015.

$$A^n_{v,v} = \text{le nombre de chemins de longueur exactement } n \text{ de } v \text{ vers } v$$

4.2.1 Avec un chemin exactement n

Une fois le nombre de chemins d'exactement n calculé, nous sommons tous les éléments de la matrice A^n ce qui nous permet d'estimer le nombre de chemins de longueur exactement n.

$$P(v,n) = \frac{A^n_{v,v}}{\sum_{i=1}^{n} \sum_{j=1}^{n} A^n_{i,j}}$$

où P est l'estimation de la probabilité de revenir en v avec un longueur de chemin n

Ce calcul est basé sur

$$P = \frac{\text{Nombre de cas favorables}}{\text{Nombre de cas possibles}}$$

4.2.2 Avec un chemin au plus n

Soit $P(v,n)$ la fonction qui permet d'estimer la probabilité de revenir en v avec une longueur de chemin n.

Afin d'estimer le probabilité de revenir en v avec un chemin au plus n, nous utilisons

$$M(v,n) = \sum_{i=1}^{n} P(v,i)$$

4.3 Parcours aléatoire du graphe

La fonction `paths(graph,v)` prend deux arguments, `graph` le graphe dans lequel l'agent parcours et `v` l'indice du sommet de départ. Nous parcourons le graphe à partir du sommet v, le choix du successeur est fait aléatoirement et ainsi de suite. Une liste de chemins allant de v à v sont retournés.

4.4 Taille du chemin le plus grand/petit

La fonction `smallestPath()` et `biggestPath()` sont des fonctions similaires dans lesquels nous traversons notre liste de chemins dans lequel nous cherchons la taille qui nous convient (respectivement la plus petite et la plus grande).

4.5 Statistiques

4.5.1 Effectif total

La fonction `Total()` prend en entrée une liste d'occurences [3], parcourt la liste d'occurence en sommant chaque effectifs de chemins, et retourne l'effectif total des chemins allant de v à v.

3. La liste d'occurences est fournie par une bibliothèque python nommée Collections, nous l'utilisons dans utils.py

4.5.2 Distribution des fréquences

La fonction `frequencyDistribution()` prend en entrée une liste d'occurences, parcourt la liste d'occurences en divisant l'effectif par l'effectif total, et retourne une liste de tuples du type `((chemin),frequence)`.

Soit N l'effectif total, x_i l'effectif du chemin i, la fréquence F_i est donné par :

$$F_i = \frac{x_i}{N}$$

est la formule utilisée pour calculer la distribution des fréquences des chemins.

4.5.3 La moyenne des chemins

La fonction `meanLenPaths(stats,tot)` prend en entrée 2 arguments, `stats` la liste statistique de type `((chemin),effectif,frequence)` et `tot` l'effectif total. La fonction Somme la multiplication la longueur de chaque chemins avec son effectif.

Soit len() une fonction qui permet de retourner la longueur d'un chemin, c_i le chemin à l'indice i, n_i l'effectif de c_i .

La moyenne de la longueur des chemins est donnée par :

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} \text{len}(c_i) \times n_i$$

4.5.4 La variance des chemins

La fonction `variance(stats,tot,mean)` est une simple traduction de la formule de la variance

$$Var = \frac{1}{N} \sum_{i=1}^{N} x_i^2 - \bar{x}^2$$

où x_i est la longueur du chemin multiplié par son effectif $\bar{x}$ est la moyenne des longueurs de chemins

5 Données de retour du programme

Le nombre de chemins de longueur n allant de v à v [4], la probabilité de revenir en v avec un chemin de longueur exactement n et la probabilité de revenir en v avec une longueur de chemin au plus n sont affiché dans le terminal

Le programme fournit un fichier `stat.txt` qui contiendra

- La taille du plus grand chemin
- La taille du plus petit chemin
- L'effectif total des chemins
- une liste de chemins de type ((chemin), effectif du chemin, frequence)

Avec ce fichier, une petite analyse peut-être fait sur le parcours de l'agent dans le graphe afin de voir quel sommet, quel est le chemin le plus emprunté par l'agent et plus encore.

4. La valeur retournée par le programme pourrait être tellement élevée que cela affiche une valeur négative

6 Utilisation du programme

Veuillez lire le fichier `LISEZMOI.TXT`

Références

[Han15] Mihai Hangiu. *Creating a matrix of arbitrary size where rows sum to 1 ?* 2015. URL : `https://stackoverflow.com/a/31364847`.

[Mau17] Peter M Maurer. "Generating strongly connected random graphs". In : *Proceedings of the International Conference on Modeling, Simulation and Visualization Methods (MSV)*. The Steering Committee of The World Congress in Computer Science, Computer ... 2017, p. 3-6.